

Bloc 1- SQL

mardi 1 juillet 2025 15:40

Bloc 1- SQL

https://colab.research.google.com/drive/1naCJS6_76DEXjhMPaRZGf4gjf-lLeCZo?usp=sharing

Révision avec windows function (pas de windows function au RNCP)

+ fin du colab API (avec Nominatim - OpenStreetMap)

<https://colab.research.google.com/drive/1UJV166bYq8m1zLYfyyKIXBEccdsxjtrQ?usp=sharing>

Révision SQL NorthWind

<https://colab.research.google.com/drive/15XJGSuCFM3VqETzOIX-ROR6rb6-TPSIk?usp=sharing>

#1. Liste des produits les plus vendus par catégorie :

```
query("""
SELECT CategoryName AS category, SUM(quantity) AS total_sales
FROM orders o
JOIN order_details AS od ON o.orderid = od.orderid
JOIN products AS p ON p.productid = od.productid
JOIN categories AS c ON p.categoryid = c.categoryid
GROUP BY CategoryName
""")
```

#2. Les cinq meilleurs clients (en terme de montant total des achats) :

```
query("""
SELECT companyname, SUM(quantity*unitprice) AS total_order
FROM customers AS c
JOIN orders AS o ON o.customerid = c.customerid
JOIN order_details AS od ON od.orderID = o.orderID
GROUP BY companyname
ORDER BY total_order DESC
LIMIT 10
""")
```

#3. Employes avec la plus grande quantité de produits vendus :

```
query("""
SELECT firstname || ' ' || lastname AS name, SUM(quantity) AS sold_qte
FROM employees AS e
JOIN orders AS o ON o.employeeid = e.employeeid
JOIN order_details AS od ON od.orderid = o.orderid
GROUP BY name
ORDER BY sold_qte DESC
LIMIT 1
""")
```

#4. Date de la première et dernière commande pour chaque client :

```
query("""
SELECT companyname,
       MIN(orderdate) OVER (PARTITION BY companyname) AS first_order,
       MAX(orderdate) OVER (PARTITION BY companyname) AS last_order
FROM customers AS c
JOIN orders AS o ON o.customerid = c.customerid
""")
```

#5. Lister les produits jamais commandés :

```
query("""
```

```

SELECT p.productname AS name,
       SUM(od.quantity) AS total_sales
FROM products AS p
LEFT JOIN order_details AS od ON p.productid = od.productid
GROUP BY p.productname
HAVING total_sales IS NULL OR total_sales = 0
""")

```

#6. Chiffres d'affaires mensuel pour la dernière année :

```

query("""
SELECT strftime("%m", orderdate) AS month, SUM(quantity*unitprice) AS ca
FROM order_details AS od
JOIN orders AS o ON o.orderid = od.orderid
WHERE strftime("%Y", orderdate) = "2023"
GROUP BY month
ORDER BY month
""")

```

#7. Produits ayant généré le plus de chiffre d'affaires, avec le total de commandes et le chiffre d'affaires généré :

```

query("""
WITH cte AS (
    SELECT productname, quantity*od.unitprice AS ca, orderid
    FROM products AS p
    JOIN order_details AS od ON od.productid = p.productid
    GROUP BY productname, orderid)
SELECT productname, SUM(ca) AS total_ca, COUNT(*) AS total_order
FROM cte
GROUP BY productname
ORDER BY total_ca DESC
""")

```

#8. Les produits les plus commandés dans chaque catégorie avec leurs quantités totales :

```

query("""
WITH cte AS (
    SELECT categoryname, productname, SUM(quantity) AS qte
    FROM categories AS c
    JOIN products AS p on p.categoryid = c.categoryid
    JOIN order_details AS od on od.productid = p.productid
    GROUP BY categoryname, productname
    ORDER BY categoryname, qte DESC),
ranks AS (
    SELECT *,
    ROW_NUMBER() OVER (PARTITION BY categoryname) AS rank
    FROM cte)
SELECT categoryname, productname, qte
FROM ranks
WHERE rank = 1
""")

```

#9. Clients qui ont passé plus de cinq commandes :

```

query("""
SELECT companyname, COUNT(orderid) AS nb_orders
FROM customers AS c
JOIN orders AS o on o.customerid = c.customerid
GROUP BY companyname
HAVING nb_orders > 5
ORDER BY nb_orders DESC
""")

```

#10. Employés qui ont traité des commandes d'une valeur totale supérieure à 50 000 \$:

```

query("""
WITH cte AS (
    SELECT firstname||' '||lastname AS name, o.orderid, SUM(quantity*unitprice) AS order_total
    FROM employees AS e

```

```

        JOIN orders AS o ON o.employeeid=e.employeeid
        JOIN order_details AS od ON od.orderid = o.orderid
        GROUP BY name, o.orderid
        HAVING order_total > 50000)
SELECT DISTINCT name FROM cte
""")

```

#11. Comparer les ventes mensuelles avec le mois précédent pour chaque produit :

```

query("""
WITH monthly_sales AS (
    SELECT strftime("%Y-%m", orderdate) AS month, productname, SUM(quantity*od.unitprice) AS ca
    FROM orders AS o
    JOIN order_details AS od ON od.orderid = o.orderid
    JOIN products AS p ON od.productid = p.productid
    GROUP BY productname, month
    ORDER BY orderdate ASC)
SELECT *
    , LAG(ca, 1) OVER(PARTITION BY productname ORDER BY month) AS previous
    , ca - LAG(ca, 1) OVER (PARTITION BY productname ORDER BY month) AS evolution
FROM monthly_sales
ORDER BY productname, month
""")

```

#12. Diviser les produits en quartiles basés sur leur prix :

```

query("""
SELECT productname, unitprice, NTILE(4) OVER() AS quartile
FROM products
ORDER BY quartile, unitprice
""")

```

#13. Calculer le cumul des ventes par jour pour chaque employé :

```

query("""
WITH cte AS (
    SELECT firstname||' '||lastname AS name, orderdate AS date, SUM(quantity*unitprice) AS ca
    FROM employees AS e
    JOIN orders AS o ON o.employeeid = e.employeeid
    JOIN order_details AS od ON od.orderid = o.orderid
    GROUP BY name, date)
SELECT name, date, SUM(ca) OVER (PARTITION BY name ORDER BY date) AS rolling_ca
FROM cte
""")

```

#14. Catégoriser les commandes en fonction de leur montant total :

```

query("""
WITH cte AS (
    SELECT orderid, SUM(quantity*unitprice) AS total
    FROM order_details
    GROUP BY orderid
    ORDER BY total),
quartile AS (
    SELECT *
    , NTILE(10) OVER(ORDER BY total) AS q
    FROM cte
    ),
first AS (
    SELECT *, ROW_NUMBER() OVER(PARTITION BY q) AS r
    FROM quartile
    )
SELECT *
FROM first
WHERE r=1
""")

```

Bloc 2 - Pandas

lundi 21 juillet 2025 17:23

Révision semaine 1

<https://colab.research.google.com/drive/18HC3TE-GOaX1qPnAJR9xGgCALhQVz-fM?usp=sharing>

Révision semaine 2 :

<https://colab.research.google.com/drive/1NGVUq8LILStWW18Y6xxmDX7db2Cvix7S?usp=sharing>

```
# Afficher les 3 premières lignes
df.head(3)

# Afficher les 6 dernières lignes
df.tail(6)

# Afficher un échantillon de 7 lignes
df.sample(7)

# Afficher les infos
df.info()

# Afficher les types de données datatypes
df.dtypes

# Afficher le nombre de chaque type
df.dtypes.value_counts()

# Afficher la forme du DataFrame un tuple (2152,12)
df.shape

# Afficher le nom des colonnes
df.columns

# Afficher les 3 premières lignes d'une colonne précise
df["Age"].head(3)

# Afficher les 4 premières lignes de 3 colonnes précises
df[["Age", "Transaction_date", "Gender"]].head(4)

# Afficher de la 2ème ligne à la 6ème lignes (rows)
df[1:6]

# pour afficher les lignes à partir de la 10 lignes
df[9:]

# pour afficher les 2 dernière lignes
df[-2:]

# supprimer 1 colonne entière (2 manières)
df.drop(columns=["Transaction_ID"], inplace = True)
# équivalent :
df.drop("Transaction_ID", axis = 1, inplace = True)

# créer une copie du DataFrame
df_col = df.copy()

# Renommer les colonnes
df.rename(columns={"Gender": "Sex", "Transaction_date": "Date"}, inplace=True)

# ajouter une nouvelle colonnes (ceci est un exemple)
df["new_col"] = df["Amount_spent"] * 100

# Modifier cellule d'un colonne Female to Woman and Male to Man
df.loc[df.Sex == "Female", "Sex"] = "Woman"
df.loc[df.Sex == "Male", "Sex"] = "Man"

# Modifier cellule d'un colonne
df['jour_affluence'] = df['jour_affluence'].replace({1: 'Mardi', 2: 'Mercredi', 3: 'Jeudi', 4: 'Vendredi', 5: 'Samedi', 6: 'Dimanche', 0: 'Lundi'})

# changer un "object" en "datetime64" d'une colonne [df.info()]
df["Date"] = df["Date"].astype("datetime64[ns]")

# changer un float64 en float32 d'une colonne
df["Referral"] = df["Referral"].astype("float32")
Ou pour changer plusieurs col en 1 seul fois
df[["fare_amount", "extra", "mta_tax", "tip_amount", "tolls_amount"]] =
df[["fare_amount", "extra", "mta_tax", "tip_amount", "tolls_amount"]].astype("float16")

# afficher le nombre de lignes dupliquées
df.duplicated().value_counts()

# Afficher les lignes dupliquées et ne garder que l'élément 'first', 'last' and False
duplicate_value = df.duplicated(keep = "first")
df.loc[duplicate_value, :]

# supprime toutes les valeur en doublons en ne gardant que la dernière occurrence
df.drop_duplicates(keep="first", inplace = True)

# afficher les colonnes avec le nombre de valeurs manquantes
df_col.isna().sum().sort_values(ascending = False)

# supprimer le "NaN" d'une colonne
df.dropna(subset = ["Employees_status"], axis = 0, inplace = True)
df.isna().sum().sort_values(ascending = False)

# supprimer 2 colonnes précises
df.drop(columns = ["new_col", "Amount_spent"], inplace = True)

# compter le nombre de "NaN" par colonne et les classer du plus grand au plus petit
df.isna().sum().sort_values(ascending = False)
```

VALEURS MAQUANTES - MISSING VALUES

Méthode 1 : suppression des colonnes manquantes

```
# remplacer les valeurs manquantes d'une colonne par "Unknown"
df["Sex"].fillna("Unknown", inplace = True)
```

```
# compte le nombre d'occurrence différente sur la col "Sex"
df.Sex.value_counts()
```

count	
Sex	
Woman	1351
Man	1121
Unknown	28

dtype: int64

Méthode 2 : remplacer les valeurs

```
# remplacer les valeurs manquantes par la moyenne de la colonne
df_mean_Am_S = df.Amount_spent.mean()
df["Amount_spent"].fillna(df_mean_Am_S, inplace = True)
# remplacer les valeurs manquantes par la médiane de la colonne ou Min et Max
df_med_Am_S = df.Amount_spent.median()
df["Amount_spent"].fillna(df_med_Am_S, inplace = True)
# remplacer les valeurs manquantes par le mode de la colonne
(mode est la valeur la plus représentée dans la colonne)
df_mode_emp_status = df.Employees_status.mode()[0]
df.Employees_status.fillna(df_mode_emp_status, inplace = True)
```

Méthode 2 : remplacer les valeurs avec la précédente ou suivante

```
# remplacer les valeur manquantes par la valeur précédente (structures temporelles)
df.Referal.fillna(method = "ffill", inplace = True)
```

```
# remplacer les valeur manquantes par la valeur suivante (structures temporelles)
df.Referal.fillna(method = "bfill", inplace = True)
```

fonction basiques pour l'analyse

```
# describe sur valeurs numerique
df.describe().T
```

```
# describe sur valeurs categorielles (textes)
df.describe(include = object).T
```

```
# Calcule de la moyenne
mean = df['age'].mean()
# Calcule de la mediane
Median = df['age'].median()
# Calculer le mode
mode = df['age'].mode().iloc[0]
# Calculer la standard deviation / Ecart-Type
std = df['age'].std()
# Calculer le Minimum
min_val = df['age'].min()
# Calculer le Maximum values
max_cal = df['age'].max()
```

```
print(f" Mean of Age : {mean}")
print(f" Median of Age : {median}")
print(f" Mode of Age : {mode}")
print(f" Standard deviation of Age : {std:.2f}")
print(f" Maximum of Age : {max_cal}")
print(f" Menimum of Age : {min_val}")
```

```
# calculer la correlation uniquement sur valeur numériques
df_col.corr(numeric_only=True)
```

```
# afficher le nombre de "Unique"
Df['age'].nunique()
```

```
# afficher les "Unique"
Df['age'].unique()
```

```
# afficher les "Unique"
Df['Sex'].value_counts()
```

count	
Sex	
Woman	1351
Man	1121
Unknown	28

dtype: int64

```
# affiche le % d'occurrence qu'il occupe sur la colonne
Df['Sex'].value_counts(normalize=True)
```

```
# classe les valeurs de la colonne
df.sort_values(by=['State_names'])
```

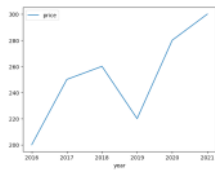
```
# Sort Values Amount_spent with ascending order
df.sort_values(by=["Amount_spent"], ascending= True)
ou
# Sort Values Amount_spent with descending order
df.sort_values(by=["Amount_spent"], ascending= False)
ou
# nsmallest
df.nsmallest(3, "Amount_spent")
Ou
# nlargest
df.nsmallest(3, "Amount_spent")
```

```
# filtrage
condition = df["Payment method"] == "PayPal"
```

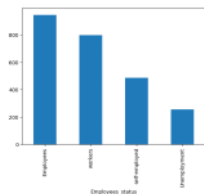
Data Visualisation

Graphique de base Pandas

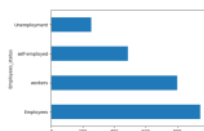
Line plot
df_line.plot(x='year', y='price')



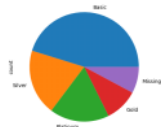
bar plot 1
df['Employees_status'].value_counts().plot(kind='bar')



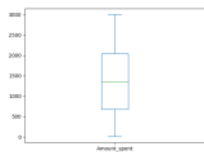
bar plot 2
df['Employees_status'].value_counts().plot(kind='barh')



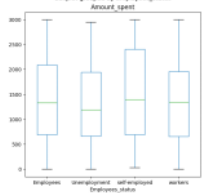
Pie plot
df['Segment'].value_counts().plot(kind='pie')



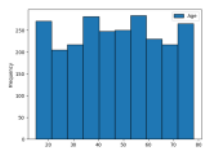
box plot / boîte à moustache
df.plot(y='Amount_spent', kind='box')



Plusieurs box plot (matplotlib)
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(6,6))
df.boxplot(by='Employees_status', column=['Amount_spent'], ax=ax, grid=False)



Histogramme (fréquence et distribution)
df.plot(y='Age', kind='hist', bins=10, edgecolor='black')



Ou
nlargest
df.nsmallest(3, "Amount_spent")

filtrage
condition = df["Payment_method"] == "PayPal"
df[condition]

filtrage avec 3 conditions
female_person = df['Gender'] == 'Female'
married_person = df['Marital_status'] == 'Married'
loc_newyork = df['State_names'] == 'New York'
we passing condition on our dataframe
df[female_person & married_person & loc_newyork]

GroupBy
Groupby
Df[["Age", "Amount_spent"]].groupby(df["Sex"]).max()

groupby multi-colonne
state_gender_res =
df[['State_names', 'Gender', 'Payment_method', 'Amount_spent']].groupby(['State_names', 'Gender', 'Payment_method']).agg(['count', 'min', 'max'])

Crosstab - tableau dynamique croisé

pd.crosstab(df.Marital_status, df.Payment_method)

Payment_method	Card	Other	PayPal
Marital_status			
Married	441	362	670
Single	308	233	498

pd.crosstab(df.Marital_status, df.Payment_method, margins=True, margins_name="Total")

Payment_method	Card	Other	PayPal	Total
Marital_status				
Married	441	362	670	1473
Single	308	233	498	1039
Total	749	595	1168	2512

crosstab avec normalize=true pour afficher un %
pd.crosstab(df.Marital_status, df.Payment_method, normalize=True, margins=True, margins_name="Total")

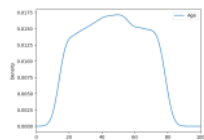
Payment_method	Card	Other	PayPal	Total
Marital_status				
Married	0.175557	0.144108	0.266720	0.586385
Single	0.122611	0.092755	0.198248	0.413615
Total	0.298169	0.236863	0.464968	1.000000

autre possibilité

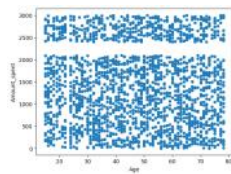
pd.crosstab(df.Marital_status, [df.Payment_method, df.Employees_status])

Payment_method	Card	Other			Paypal			
Employees_status	Employees	Unemployment	self-employed	workers	Employees	Unemployment	self-employed	workers
Marital_status								
Married	179	38	81	136	127	41	79	114
Single	115	37	57	96	95	17	44	76

```
KDE plot  
df.plot(y='Age',xlim=(0, 100),kind='kde')
```



```
Scatter plot / nuage de points  
df.plot(x='Age',y='Amount_spent',kind='scatter')
```



Bloc 3 -MACHINE LEARNING

https://colab.research.google.com/drive/1JULy4pUVtn5NmKUtp4H8o3WFTJIS_TLb?usp=sharing

Très intéressant : EDA, prepa
https://colab.research.google.com/drive/1QsFMUH3mLWuv3-qwVZoQ_n279-CLxcSQ?usp=sharing

Apprentissage Supervisé

Régression (matrice de corrélation)

- Régression Linéaire, univarié et multivarié
 - Exemple Linear regression
 - Exemple Decision tree regressor
 - Exemple K-nearest neighbors regressor
 - Métriques d'évaluation

Racine carré de l'Erreur quadratique moyenne (RMSE) : Le RMSE mesure la moyenne des erreurs au carré entre les valeurs prédites et les valeurs réelles. Il donne une indication de l'écart moyen entre les prédictions et les véritables valeurs de sortie. Une valeur de RMSE plus proche de zéro indique une meilleure performance du modèle.

Erreur absolue moyenne (MAE) ou (MAPE %) : Le MAE mesure la moyenne des erreurs absolues entre les valeurs prédites et les valeurs réelles. Il représente la magnitude moyenne des erreurs de prédiction, sans tenir compte de leur direction. Une valeur de MAE plus proche de zéro indique une meilleure performance du modèle.

Coefficient de détermination (R²) : Le coefficient de détermination, également appelé R², mesure la proportion de la variance totale des valeurs de sortie qui est expliquée par le modèle. Il indique la capacité du modèle à ajuster les données. Il permet d'évaluer la qualité d'ajustement du modèle aux données observées. Il indique la proportion de la variance totale de la variable dépendante qui est expliquée par le modèle de régression. Plus précisément, le R² mesure la proportion de la variation de la variable dépendante (y) qui peut être prédite à l'aide des variables indépendantes (x) incluses dans le modèle de régression. Il se situe entre 0 et 1, où 0 signifie que le modèle n'explique aucune variation et 1 signifie qu'il explique toute la variation de la variable dépendante.

Erreur relative moyenne (MRE) : Le MRE est une mesure relative qui exprime l'erreur moyenne en pourcentage par rapport à la valeur réelle. Il fournit une indication de l'erreur moyenne relative du modèle. Une valeur de MRE plus proche de zéro indique une meilleure performance du modèle.

Classification (matrice de confusion)

- KNN (K-nearest neighbors classifier)
- Logistic regression
- Decision tree classifier
- Métriques d'évaluation

Matrice de confusion:
la matrice de confusion permet d'évaluer les performances d'un modèle de classification en comparant les prédictions du modèle avec les valeurs réelles. Elle fournit des informations sur les vrais positifs, les vrais négatifs, les faux positifs et les faux négatifs, qui peuvent être utilisées pour calculer des métriques d'évaluation telles que la précision, le rappel et le F1-score.

Précision (Precision): La précision mesure la proportion d'observations positives prédites correctement parmi toutes les observations positives prédites par le modèle. En d'autres termes, c'est le nombre de vrais positifs divisé par la somme des vrais positifs et des faux positifs. Une précision plus élevée indique que le modèle fait moins d'erreurs en classant les observations comme positives.

Rappel (Recall) : Le rappel, également appelé sensibilité ou taux de vrais positifs, mesure la proportion d'observations positives prédites correctement parmi toutes les observations réellement positives. C'est le nombre de vrais positifs divisé par la somme des vrais positifs et des faux négatifs. Un rappel plus élevé indique que le modèle est capable de détecter un plus grand nombre d'observations positives.

F1-score : Le F1-score est une mesure combinée de la précision et du rappel. Il est calculé comme la moyenne pondérée harmonique de la précision et du rappel. Le F1-score est souvent utilisé lorsque vous voulez trouver un équilibre entre la précision et le rappel. Une valeur plus élevée du F1-score indique un meilleur équilibre entre la précision et le rappel.

Exactitude (Accuracy) : La précision mesure la proportion d'observations correctement classées parmi toutes les observations. C'est le nombre total de prédictions correctes (vrais positifs et vrais négatifs) divisé par le nombre total d'observations. La précision est une mesure générale de la performance du modèle, mais elle peut être trompeuse si les classes sont déséquilibrées.

Score AUC-ROC : L'aire sous la courbe ROC (Receiver Operating Characteristic) est une mesure qui évalue la capacité du modèle à distinguer entre les classes. La courbe ROC représente le taux de vrais positifs en fonction du taux de faux positifs à différents seuils de classification. Plus l'aire sous la courbe ROC est proche de 1, meilleur est le modèle pour distinguer les classes.

NLP Natural Language Processing

- Tokenization avec Spacy

Apprentissage Non-Supervisé

Clustering

- K-means
- Métriques d'évaluation

Inertie intra-cluster : Elbow la cohésion des points à l'intérieur des clusters (/l\ outliers).
La méthode du coude évalue la variation de l'inertie intra-cluster en fonction du nombre de clusters. L'inertie intra-cluster mesure la cohésion des points à l'intérieur de chaque cluster. Une valeur faible d'inertie indique que les points à l'intérieur d'un cluster sont similaires entre eux. Dans la méthode du coude, on trace un graphique du nombre de clusters en fonction de l'inertie intra-cluster. L'idée est de trouver le point du graphique où l'ajout d'un cluster supplémentaire n'apporte qu'une amélioration marginale de l'inertie intra-cluster. Ce point est appelé "point du coude". Il est généralement considéré comme le nombre optimal de clusters.

Inertie inter-cluster : Silhouette la cohésion de séparation des clusters (le point est-il dans le bon cluster ?).
La méthode de la silhouette évalue à la fois la cohésion et la séparation des clusters. Elle calcule la silhouette pour chaque point de données, qui mesure à quel point ce point est bien classé dans son propre cluster par rapport aux autres clusters. La silhouette varie de -1 à 1, où une valeur proche de 1 indique que le point est bien classé, une valeur proche de 0 indique une incertitude, et une valeur proche de -1 indique que le point est mal classé. Dans la méthode de la silhouette, on calcule la silhouette moyenne pour chaque nombre de clusters. On recherche le nombre de clusters qui maximise la silhouette moyenne, ce qui indique une bonne séparation des clusters.

Grid Search CV (pour trouver les meilleurs hyperparamètre)

Cross Validation

Standardisation avec StandardScaler

ACP

Bonus : Lapz Predict

- Score r^2 de 0.79 la variance explique le modèle ou généralise bien le modèle
- Pipeline automatise le traitement conseillé d'en mettre 1
- StandardScaler : met à l'échelle toutes les données numériques (moyenne de 0 avec écart type de 1), ne sert à rien dans un arbre de décision
- Un "Encoder" sert à encoder les textes en valeurs numériques (à mettre dans un pipeline pour automatiser
(from sklearn.preprocessing import OneHotEncoder)
- stratify=y , conserve les proportions des classes dans les deux sous-ensembles. Très utile pour des classes déséquilibrées (ex : 90% de 0, 10% de 1)

Les résultats R2 montrent-ils que le modèle est efficace ?
Oui, le modèle est efficace avec un R2 qui explique 91% de la variance des données de test, et une MAE de 9.3k pour une médiane de 107K, soit une erreur de 9.7% en moyenne

Les résultats R2 montrent-ils que le modèle est efficace ?
Oui, le modèle est efficace avec un R2 qui explique 91% de la variance des données de test, et une MAE de 9.3k pour une médiane de 107K, soit une erreur de 9.7% en moyenne.

Voir collab : https://colab.research.google.com/drive/1tydSU_DdL_Th3-SZSNuw3Q8wom7wnDXx2uspssharing

Exemple code ML (sensiblement toujours la même chose)

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import accuracy_score
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error,
mean_absolute_percentage_error, root_mean_squared_error

# Charger un jeu de données exemple
data = load_breast_cancer()
X = data.data
y = data.target

# Séparer en train / test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42, stratify=y)

# Créer le pipeline
pipeline = Pipeline(steps=[
    ('scaler', StandardScaler()), # Étape de standardisation
    ('classifier', LogisticRegression()) # Modèle de classification
])

# Entraîner le pipeline
pipeline.fit(X_train, y_train)

# Prédire
y_pred = pipeline.predict(X_test)

# metrics en fonction du modèle utilisé (supervisé ou non supervisé)

# metrics supervisé
R2 = pipeline.score(X_test, y_test)
mae = mean_absolute_error(y_true=y_test, y_pred = y_pred)
mape = mean_absolute_percentage_error(y_true=y_test, y_pred = y_pred)
rmse = root_mean_squared_error(y_true=y_test, y_pred = y_pred)
print(f"R2 : {R2}, MAE : {mae}, MAPE : {mape*100} %, RMSE : {rmse}")

# metrics non-supervisé
cm = confusion_matrix(y_true = y_test, y_pred=y_pred, labels=pipeline.classes_)
disp = ConfusionMatrixDisplay(cm, display_labels=pipeline.classes_)
disp.plot()
plt.show()
print(classification_report(y_true = y_test, y_pred = y_pred))
```

Exemple code ML ColumnTransformer et standardScaler :

```
ColumnTransformer pour les valeurs catégorielles (textes)
StandardScaler pour les valeurs numériques

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.metrics import accuracy_score, classification_report

# Exemple de dataset fictif
data = pd.DataFrame({
    "age": [25, 45, 35, 50, 23, 52, 40, 60],
    "sexe": ["H", "F", "F", "H", "F", "H", "H", "F"],
    "salaire": [30000, 60000, 40000, 80000, 25000, 90000, 70000, 50000],
    "ville": ["Paris", "Lyon", "Lyon", "Paris", "Marseille", "Lyon", "Paris", "Marseille"],
    "cible": [0, 1, 0, 1, 0, 1, 1, 0]
})

# 1. Définir X (features) et y (target)
X = data.drop(columns="cible")
y = data["cible"]

# 2. Définir les colonnes numériques et catégorielles
colonnes_numeriques = ["age", "salaire"]
colonnes_categorielles = ["sexe", "ville"]

# 3. Créer le préprocesseur avec ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), colonnes_numeriques),
        ("cat", OneHotEncoder(), colonnes_categorielles)
    ]
)
```

Standardisation avec StandardScaler

ACP

Bonus : Lazy Predict

https://colab.research.google.com/drive/1tydSU_OdL_Th3-S2SNuw3QBwom7wqDXx?usp=sharing

Régression

```
# Définir X et y
X = df.drop('Profit', axis = 1)
y = df['Profit']

# train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=.75, random_state=42, stratify=y)

# Afficher la matrice de corrélation uniquement valeurs numériques
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
cm = df.select_dtypes('number').corr()
mask = np.triu(np.ones_like(cm))
sns.heatmap(cm, mask=mask, annot=True, fmt=".2f", cmap='magma', center=0)
plt.show()

# Créez un pipeline pour encoder la colonne 'State'
et normaliser les colonnes numériques
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LinearRegression
```

```
cat_col = X.select_dtypes(include='object').columns.tolist()
num_col = X.select_dtypes(include='number').columns.tolist()
```

```
model = LinearRegression()

preprocessing = ColumnTransformer([
    ('cat', OneHotEncoder(), cat_col),
    ('scaler', StandardScaler(), num_col)
])
```

```
pipe = Pipeline(steps=[
    ('preprocessing', preprocessing),
    ('model', model)
])
```

Pipe # pour afficher le pipeline

```
# Entraînez le modèle et affichez son score R2 sur le train et le test
pipe.fit(X_train, y_train)
```

```
R2train = pipe.score(X_train, y_train)
R2test = pipe.score(X_test, y_test)
```

```
print(f"R2 train : {R2train}\tR2 test : {R2test}")
```

```
# Affichez la valeur médiane du profit
# Affichez les scores de RMSE, MAE et MAPE (mean absolute percentage error)
from sklearn.metrics import root_mean_squared_error, mean_absolute_error,
mean_absolute_percentage_error
```

```
y_pred = pipe.predict(X_test)
```

```
med = y.median()
mae = mean_absolute_error(y_test, y_pred)
rmse = root_mean_squared_error(y_test, y_pred)
mape = mean_absolute_percentage_error(y_test, y_pred)
```

```
print(f"Mediane = {med}\tRMSE = {rmse:.2f}\tMAE = {mae:.2f}\tMAPE = {mape*100:.2f}")
```

Classification sans pipeline ni ColumnTransformer

```
# Afficher la matrice de corrélation uniquement valeurs numériques
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
cm = df.select_dtypes('number').corr()
mask = np.triu(np.ones_like(cm))
sns.heatmap(cm, mask=mask, annot=True, fmt=".2f", cmap='magma', center=0)
plt.show()
```

```
# Définir X et y
X = df.drop('Outcome', axis = 1)
y = df['Outcome']
```

```
# train_test_split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=.75, random_state=42, stratify=y)
```

```
# Entraînez un modèle de Random Forest et affichez la justesse train/test
# Equilibrez les classes (class_weight)
from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier(class_weight="balanced")
```

```
model.fit(X_train, y_train)
```

```
score_train = model.score(X_train, y_train)
score_test = model.score(X_test, y_test)
```

```
print(f"Accuracy train = {score_train:.2f}\tAccuracy test : {score_test:.2f}")
```

```
# S'il y a overfit, n'hésitez pas à modifier le paramètre max_depth
from sklearn.ensemble import RandomForestClassifier
```

```
transformers=[
    ("num", StandardScaler(), colonnes_numeriques),
    ("cat", OneHotEncoder(), colonnes_categorielles)
]
```

```
# 4. Créer le pipeline complet
pipeline = Pipeline(steps=[
    ("preprocessing", preprocessor),
    ("model", RandomForestClassifier(random_state=42))
])
```

```
# 5. Séparer les données en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)
```

```
# 6. Entraîner le modèle
pipeline.fit(X_train, y_train)
```

```
# 7. Prédiction sur le jeu de test
y_pred = pipeline.predict(X_test)
```

```
# 8. Évaluation
print("Accuracy :", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
# 9. Faire une prédiction sur une nouvelle observation
nouvelle_donnee = pd.DataFrame({
    "age": [30],
    "sexe": ["F"],
    "salaire": [45000],
    "ville": ["Lyon"]
})
```

```
prediction = pipeline.predict(nouvelle_donnee)
print("Prédiction pour la nouvelle donnée :", prediction[0])
```

```

model = RandomForestClassifier(class_weight="balanced", max_depth=3)

model.fit(X_train, y_train)

score_train = model.score(X_train, y_train)
score_test = model.score(X_test, y_test)

print(f"Accuracy train = {score_train:.2f}\tAccuracy test : {score_test:.2f}")

# Affichez la matrice de confusion
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(cm)
disp.plot()
plt.show()

# Affichez les rapport de classification
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))

```

Révision Régression Linéaire + Code Maelle

mardi 15 juillet 2025 16:38

<https://colab.research.google.com/drive/10PgusnmDEkGe3dvcv-7lgDUBitVYAQcrs?usp=sharing>

Pipeline

Encoder : est un type spécifique de transformer utilisé pour convertir des données catégorielles en format numérique.

Transformer : est un objet qui transforme les données d'entrée (souvent brutes ou hétérogènes) en données prêtes à être utilisées par un algorithme d'apprentissage

Métrics : R², MAE, MSE, RMSE ...

Scatter plot nuage de points

Box plot boîte à moustache

Métrique pour un régression linéaire ou régression logistique R², MAE, MSE et RMSE

RNCP BLANC - Code Maelle

Régression Linéaire

```
df = pd.read_csv("https://raw.githubusercontent.com/polaternez/predicting-used-car-price-UK/refs/heads/master/data/uk_used_car.csv")
```

```
df.head()
```

```
df.shape
```

```
df.info()
```

```
df.dtypes
```

```
df.describe(include='number').T
```

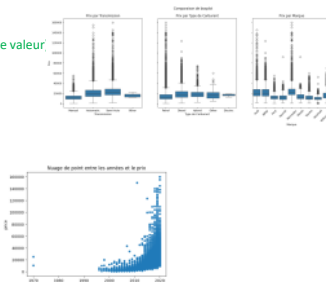
```
numeric_df = df.select_dtypes(include=['number'])
correlation_matrix = numeric_df.corr()
correlation_matrix
```

matrice de corrélation

```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
mask = np.triu(correlation_matrix)
plt.figure(figsize=(10,8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f',
mask=mask)
plt.title('Matrice de corrélation')
plt.show()
```

Box plot bivarié (plusieurs plot sur le meme graph dans la meme unité y toujours meme valeur

```
fig, axes = plt.subplots(1,3, figsize=(18,6), sharey=True)
fig.suptitle('Comparaison de boxplot')
sns.boxplot(data=df, x='transmission', y='price', ax=axes[0])
axes[0].set_title('Prix par Transmission')
axes[0].set_xlabel('Transmission')
axes[0].set_ylabel('Prix')
sns.boxplot(data=df, x='fuelType', y='price', ax=axes[1])
axes[1].set_title('Prix par Type de Carburant')
axes[1].set_xlabel('Type de Carburant')
sns.boxplot(data=df, x='brand', y='price', ax=axes[2])
axes[2].set_title('Prix par Marque')
axes[2].set_xlabel('Marque')
axes[2].tick_params(axis='x', rotation=45)
fig.show()
```



Nuage de points

```
df.plot(x='year', y='price', kind='scatter')
plt.title('Nuage de point entre les années et le prix')
plt.show()
```



Déterminer les variables

```
X = df.drop('price', axis=1)
y = df['price']
```

Train test split (pas de stratify=y dans régression linéaire)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=.8,
random_state=42)
```

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LinearRegression
```

```
# Séparation des données quantitatives et qualitatives sur X_reg
cat_col = X.select_dtypes(include=['object']).columns.tolist()
numeric_col = X.select_dtypes(include=['number']).columns.tolist()
```

Initialisation du modèle

```
model = LinearRegression()
```

```
# Création du transformer
transformer = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_col),
    ('scaler', StandardScaler(), numeric_col)
])
```

Création de la pipeline

```
pipe = Pipeline(steps=[
    ('transformer', transformer),
    ('model', model)
])
```

affichage du Pipeline

```
pipe
```

Entraînement du modèle

```
pipe.fit(X_train, y_train)
```

Calcul et affichage des R² train et R² test

```
R2train = pipe.score(X_train, y_train)
R2test = pipe.score(X_test, y_test)
print(f"R2 train : {R2train:.4f}\tR2 test : {R2test:.4f}")
```

Régression

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error,
mean_absolute_percentage_error, root_mean_squared_error
```

importation BDD

```
df = pd.read_csv("/content/medical_cost_insurance.csv")
```

```
df.head()
```

```
df.isna().sum()
```

```
df.info()
```

```
df.describe()
```

```
Duplicate().sum() ??? Et drop duplicate() ??
```

```
df.smoker.unique()
```

```
df.sex.unique() # données catégorielle à encoder au ft numeric
```

```
df.region.unique() # données catégorielle à encoder au ft numeric
```

Régression linéaire

```
X = df[df.columns[:-1]]
y = df.charges
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size
=.8, random_state=42)
```

```
num_cols = df.iloc[:,0:5].columns
cat_cols = ['region']
```

```
transformer = ColumnTransformer([
    ('num', StandardScaler(), num_cols), # Transformer
    ('cat', OneHotEncoder(), cat_cols)
])
pipeline = Pipeline(steps=[
    ('transformer', transformer),
    ('model', LinearRegression())
])
```

```
pipeline.fit(X_train, y_train)
Pipeline
```

```
y_pred = pipeline.predict(X_test)
pipeline.score(X_test, y_test)
```

Metrics

```
from sklearn.metrics import r2_score,
mean_squared_error, mean_absolute_error,
mean_absolute_percentage_error, root_mean_squared_error
```

```
mae = mean_absolute_error(y_true=y_test, y_pred = y_pred)
mape = mean_absolute_percentage_error(y_true=y_test, y_pred =
y_pred)
rmse = root_mean_squared_error(y_true=y_test, y_pred = y_pred)
print(f"MAE : {mae}, MAPE : {mape*100} %, RMSE : {rmse}")
```

Régression RandomForestRegressor

```
from sklearn.ensemble import RandomForestRegressor
```

```
pipeline = Pipeline(steps=[
    ('transformer', transformer),
    ('model', RandomForestRegressor())
])
pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
R2 = pipeline.score(X_test, y_test)
mae = mean_absolute_error(y_true=y_test, y_pred = y_pred)
mape = mean_absolute_percentage_error(y_true=y_test, y_pred =
y_pred)
rmse = root_mean_squared_error(y_true=y_test, y_pred = y_pred)
print(f"R2 : {R2}, MAE : {mae}, MAPE : {mape*100} %, RMSE : {rmse}")
```

scatter + droite de regression

```
fig, axes = plt.subplots(2,1)
sns.regplot(ax=axes[0], data =df, x='age', y = 'charges')
axes[0].set_title("Age vs charge")
sns.regplot(ax=axes[1], data = df, x = 'bmi', y = 'charges')
axes[1].set_title("BMI vs charge")
plt.tight_layout()
plt.show()
```

box plot - boîte à moustache

```
sns.boxplot(data=df, x = 'smoker', y = 'charges')
plt.show()
```

```
sns.boxplot(data=df, x = 'children', y = 'charges')
plt.show()
```

```

# Calcul et affichage des R2 train et R2 test
R2train = pipe.score(X_train, y_train)
R2test = pipe.score(X_test, y_test)
print(f"R2 train : {R2train:.4f}\tR2 test : {R2test:.4f}")

# Affichage des métriques du modèle
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error,
mean_absolute_percentage_error, root_mean_squared_error

# Prédiction et autres métriques
y_pred = pipe.predict(X_reg_test)
print("---- Métriques de performance du modèle ----")

# Median
# med = y_reg.median()
# print(f"La médiane est de {med}")
# R-squared

r2 = r2_score(y_reg_test, y_pred)
print(f"R2 (Coefficient de détermination) : {r2:.4f}")

# Mean Absolute Error (MAE)
mae = mean_absolute_error(y_reg_test, y_pred)
print(f"Mean Absolute Error (MAE) : {mae:.4f}")

# Mean Absolute Error (MAPE) (plus parlant)
mae = mean_absolute_percentage_error(y_reg_test, y_pred)
print(f"Mean Absolute Error (MAE) : {mae:.4f}")

# Mean Squared Error (MSE)
mse = mean_squared_error(y_reg_test, y_pred)
print(f"Mean Squared Error (MSE) : {mse:.4f}")

# Root Mean Squared Error (RMSE)
rmse = root_mean_squared_error(y_reg_test, y_pred)
print(f"Root Mean Squared Error (RMSE) : {rmse:.4f}")

# Affichage de la distribution des erreurs

# 1 Calculer les résidus (erreurs) directement sur l'échelle log-transformée
residus = y_test - y_pred

# Afficher l'histogramme des résidus
plt.figure(figsize=(10, 6))
sns.histplot(residus, kde=True, bins=50) # 'kde=True' ajoute une courbe
d'estimation de densité
plt.title("Distribution des Erreurs (Résidus) ")
plt.xlabel('Erreurs (Vrai Prix - Prix Prédit)')
plt.ylabel('Fréquence')
plt.axvline(x=0, color='red', linestyle='--', label='Erreur nulle')
plt.axvline(x=residus.mean(), color='green', linestyle=':', label=f'Moyenne des
erreurs: {residus.mean():.4f}')
plt.legend()
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

```

```

sns.boxplot(data=df, x = 'children', y = 'charges')
plt.show()

```

```

sns.boxplot(data=df, x = 'region', y = 'charges')
plt.show()

```

Révision Classification + code Maelle

mardi 15 juillet 2025 16:07

<https://colab.research.google.com/drive/10PgusnmDEkGe3dcv-7lgDubitVYAQcrs?usp=sharing>

Métrique pour une classification "RandomForest" matrice de confusion et classification report

RNCP BLANC - Code Maelle

```
Classification
df = pd.read_csv('https://raw.githubusercontent.com/zainhaidar16/Binary-Classification-with-a-Bank-Churn-Dataset/refs/heads/main/Data/train.csv')

df.head()

df.shape

df.info()

df.dtypes

df.describe().T

df.isna().sum()

# affiche le diagramme de la colonne à prédire (permet de de voir si la variable est équilibrée ou pas)
df.Exited.value_counts().plot(kind='bar')
plt.show()

# matrice de corrélation sur les valeurs numériques
numeric_df = df.numeric.select_dtypes(include=['number'])
cm = numeric_df.corr()
Print(cm)
mask = np.triu(cm)
plt.figure(figsize=(10,8))
sns.heatmap(correlation_matrix1, annot=True, cmap='coolwarm', fmt='.2f',
mask=mask)
plt.title('Matrice de corrélation')
plt.show()

# Ensemble d'entrainement et train_test_split
X = df.drop('Exited', axis=1)
y = df['Exited']
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=.8,
random_state=42, stratify=y) # stratify assure que la répartition des classes
(ou catégories) de la variable cible y est la même dans les ensembles
d'entrainement et de test

from sklearn.linear_model import LogisticRegression

# Séparation des données quantitatives et qualitatives
cat_col = X.select_dtypes(include=['object']).columns.tolist()
numeric_col = X.select_dtypes(include=['number']).columns.tolist()

# Initialisation du modèle
model = LogisticRegression(class_weight="balanced", max_iter=1000)
# class_weight="balanced" : Gérer le déséquilibre des classes
# max_iter=1000 : Contrôler le nombre d'itérations d'entrainement

# Création du transformer

transformer = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_col),
    ('scaler', StandardScaler(), numeric_col)
])

# OneHotEncoder code les variables catégorielles en valeurs numériques " comme rouge, bleu, vert"

rouge    bleu    vert
1         0         0
0         1         0
0         0         1

# StandardScaler est un transformateur de données numériques qui applique une standardisation: il
centre et réduit les données pour qu'elles aient :
une moyenne (μ) = 0
un écart-type (σ) = 1

# Création de la pipeline
pipeline = Pipeline(steps=[
    ('transformer', transformer),
    ('model', model)
])

pipeline

# Entrainement et prédiction du modèle
pipeline.fit(X_train, y_train)

y_pred = pipeline.predict(X_test)

score_train = pipeline.score(X_train, y_train)
score_test = pipeline.score(X_test, y_test)
print(f"Accuracy train = {score_train:.2f}\tAccuracy test : {score_test:.2f}")

# Métrique pour définir la performance du modèle
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))

# Matrice de confusion
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
cm = confusion_matrix(y_test, y_pred, labels=pipeline_SODG.classes_)
disp = ConfusionMatrixDisplay(cm, display_labels=pipeline_SODG.classes_)
disp.plot()
plt.show()
```

Classification

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv(".....", sep=".....")
df.head()

df.isna().sum()

df.info()

df.describe().T

# Transforme valeur texte categorielle en chiffre
df["target_cat"] = df.Target.map({'Dropout' : 0, 'Graduate' :2, 'Enrolled':1})

# factorise une colonne
df_titanic['Sex_facto'] = df_titanic['Sex'].factorize()[0]

correlations = df[cols].corr()['target_cat'].sort_values()
cols_to_keep = list(correlations[abs(correlations)>.2].index)
cols_to_keep.remove("target_cat")
cols_to_keep

X = df[cols_to_keep]
y = df["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size = .8, random_state = 42,
stratify=y)

# choix du modèle 1 KNeighborsClassifier
from sklearn.neighbors import KNeighborsClassifier

pipe = Pipeline(steps=[
    ('scaler',StandardScaler()),
    ('model',KNeighborsClassifier(n_neighbors=11))
])
pipe.fit(X_train, y_train)
pipe

y_pred = pipe.predict(X_test)

# Affichage matrice de confusion
cm = confusion_matrix(y_true = y_test, y_pred=y_pred, labels=pipe.classes_)
disp = ConfusionMatrixDisplay(cm, display_labels=pipe.classes_)
disp.plot()
plt.show()

# Affichage rapport de classification
print(classification_report(y_true = y_test, y_pred = y_pred))

# Modèle 2 RandomForestClassifier
from sklearn.ensemble import RandomForestClassifier
pipeRFC = Pipeline(steps=[
    ('scaler',StandardScaler()),
    ('model',RandomForestClassifier(n_estimators=50, class_weight="balanced"))
])
pipeRFC.fit(X_train, y_train)

y_pred = pipeRFC.predict(X_test)

cm = confusion_matrix(y_true = y_test, y_pred=y_pred, labels=pipeRFC.classes_)
disp = ConfusionMatrixDisplay(cm, display_labels=pipeRFC.classes_)
disp.plot()
plt.show()
print(classification_report(y_true = y_test, y_pred = y_pred))

# Modèle 3 LogisticRegression
from sklearn.linear_model import LogisticRegression
pipe_LRC = Pipeline(steps=[
    ('scaler',StandardScaler()),
    ('model',LogisticRegression(class_weight="balanced", max_iter=1000)
])
pipe_LRC.fit(X_train, y_train)
y_pred = pipe_LRC.predict(X_test)
cm = confusion_matrix(y_true = y_test, y_pred=y_pred, labels=pipe_LRC.classes_)
disp = ConfusionMatrixDisplay(cm, display_labels=pipe_LRC.classes_)
disp.plot()
plt.show()
print(classification_report(y_true = y_test, y_pred = y_pred))

# Modèle 4 SGDClassifier
from sklearn.linear_model import SGDClassifier
pipe_SODG = Pipeline(steps=[
    ('scaler',StandardScaler()),
    ('model',SGDClassifier(loss="log_loss", class_weight="balanced"))
])
pipe_SODG.fit(X_train, y_train)
y_pred = pipe_SODG.predict(X_test)
cm = confusion_matrix(y_true = y_test, y_pred=y_pred, labels=pipe_SODG.classes_)
disp = ConfusionMatrixDisplay(cm, display_labels=pipe_SODG.classes_)
disp.plot()
plt.show()
print(classification_report(y_true = y_test, y_pred = y_pred))

# prédiction sur des Nouvelles données
new_data_DTC = [[1, 2, 5, 4]]
new_pred_DTC = model_DTC.predict(new_data_DTC)
print(f"Prédiction : {new_pred_DTC[0]}")
```

Révision API

lundi 21 juillet 2025 10:58

Exercice API Nominatim - Géocodage d'adresse

Objectif :

Utiliser l'API Nominatim d'OpenStreetMap pour récupérer les coordonnées GPS (latitude et longitude) d'une adresse donnée.

Consignes :

1. Implémenter une fonction `get_location(address: str) -> dict` qui prend en paramètre une adresse (chaîne de caractères).`
 2. Cette fonction doit interroger l'API Nominatim via une requête HTTP GET et récupérer la réponse au format JSON.
 3. Extraire et retourner un dictionnaire contenant au minimum :
 - 'display_name' : le nom complet normalisé du lieu retourné par l'API,
 - 'lat' : la latitude,
 - 'lon' : la longitude.
 4. Gérer le cas où aucune donnée n'est retournée (retourner un dictionnaire vide).
 5. Utiliser un header User-Agent approprié pour respecter la politique d'utilisation de l'API.
- Exemple d'adresse à tester : "10 Downing Street, London"

```
import requests
from pprint import pprint

def get_location(address: str) -> dict:
    """
    Interroge l'API Nominatim pour obtenir les coordonnées GPS d'une adresse.
    Retourne un dictionnaire contenant 'display_name', 'lat' et 'lon'.
    """
    url = "https://nominatim.openstreetmap.org/search"

    params = {
        'q': address,          # Adresse à géocoder
        'format': 'json',      # Format de réponse
        'limit': 1              # On ne récupère qu'un seul résultat (le plus pertinent)
    }

    headers = {
        'User-Agent': 'RomainDataAnalyst/1.0 (forexample@example.com)' # Obligatoire selon
les CGU de Nominatim
    }

    try:
        response = requests.get(url, params=params, headers=headers)
        response.raise_for_status() # Lève une exception si erreur HTTP
        data = response.json()
        if not data:
            return {}
        result = data[0] # Premier résultat le plus pertinent
        return {
            'display_name': result.get('display_name', ''),
            'lat': result.get('lat', ''),
            'lon': result.get('lon', '')
        }
    except requests.RequestException as e:
        print(f"Erreur lors de la requête HTTP : {e}")
        return {}

# Exemple d'utilisation
```

```
adresse = "10 Downing Street, London"  
resultat = get_location(adresse)  
print("Résultat formaté :")  
pprint(resultat)
```

Fichier EDA pré-rempli

https://colab.research.google.com/drive/1bcVxXJtMiqIZH_6j9cmEf5chQrYd9eFP?usp=sharing

1 - Analyse exploratoire des données (EDA)

Tâche	Description	Fonctions / Méthodes
Aperçu du DataFrame	Dimensions, premières lignes, colonnes	df.shape, df.head(), df.columns, df.info()
Types de données	Vérifier si les types sont corrects	df.dtypes
Statistiques descriptives	Moyenne, médiane, écart-type, etc.	df.describe()
Valeurs uniques	Détecter les colonnes catégorielles	df['colonne'].unique(), df['colonne'].nunique()
Distribution des variables	Voir la répartition des valeurs	df['colonne'].value_counts(ascending=False)
Visualisation simple	Comprendre la tendance générale	df.hist(), df.boxplot(), df['colonne'].plot(kind='bar')

2 - Nettoyage des données

Tâche	Description	Fonctions
Renommer les colonnes	Pour plus de clarté	df.rename(columns={ 'AncienNom': 'NouveauNom' })
Nettoyage	Corriger types, doublons, colonnes	dropna, drop_duplicates, astype
Valeurs manquantes / Outliers	Corriger ou supprimer	fillna, quantile, boxplot
Supprimer les colonnes inutiles	Trop de NaN, ou pas utiles pour l'analyse	df.drop(columns=['col1', 'col2'])
Supprimer les doublons	Nettoyage général	df.drop_duplicates()
Corriger les types	Dates, booléens, entiers	pd.to_datetime(df['date']), df['colonne'].astype(int)
Normaliser les textes	Minuscules, accents, etc.	df['col'].str.lower().str.strip()

3 - Traitement des valeurs manquantes et des outliers

Situation	Solution	Méthodes
< 5% manquants	Suppression	df.dropna() ou df.dropna(subset=['colonne'])
Donnée continue (âge, prix...)	Remplir par moyenne ou médiane	df['colonne'].fillna(df['colonne'].mean()) ou .median()
Donnée catégorielle	Remplir par valeur fréquente	df['colonne'].fillna(df['colonne'].mode()[0])
Trop de valeurs manquantes (> 50%)	Supprimer la colonne	df.drop(columns=['colonne'])

Outliers (valeurs aberrantes)

```
Q1 = df['colonne'].quantile(0.25)
Q3 = df['colonne'].quantile(0.75)
IQR = Q3 - Q1
limite_basse = Q1 - 1.5 * IQR
limite_haute = Q3 + 1.5 * IQR
outliers = df[(df['colonne'] < limite_basse) | (df['colonne'] > limite_haute)]
```

Visualisation :

- df.boxplot(column='colonne')

Cas	Solution
Si valeur est une erreur (ex: 1000 ans pour un âge)	Supprimer
Si cela fausse la moyenne	Utiliser médiane plutôt que moyenne

4 - Export en csv

```
df.to_csv("nyc_clean.csv", index=False)
```

Outliers

Supprimer les valeurs aberrantes sur 1 col numérique :

```
# Exemple sur la colonne 'montant'
Q1 = df['montant'].quantile(0.25)
Q3 = df['montant'].quantile(0.75)
IQR = Q3 - Q1
```

```
# Filtrage du DataFrame (on garde les valeurs dans l'intervalle acceptable)
df = df[(df['montant'] >= Q1 - 1.5 * IQR) & (df['montant'] <= Q3 + 1.5 * IQR)]
```

Remplacer les valeurs aberrantes par la médiane sur 1 col numérique :

```
Q1 = df['montant'].quantile(0.25)
Q3 = df['montant'].quantile(0.75)
IQR = Q3 - Q1
# Définir les bornes
borne_basse = Q1 - 1.5 * IQR
borne_haute = Q3 + 1.5 * IQR
# Remplacer les outliers par la médiane
mediane = df['montant'].median()
df.loc[(df['montant'] < borne_basse) | (df['montant'] > borne_haute), 'montant'] = mediane
```

Supprimer les valeurs aberrantes sur toutes les colonnes numérique :

```
colonnes_numeriques = df.select_dtypes(include='number').columns
```

```
for col in colonnes_numeriques:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    df = df[(df[col] >= Q1 - 1.5 * IQR) & (df[col] <= Q3 + 1.5 * IQR)]
```

Remplacer les valeurs aberrantes par la médiane sur toutes les colonnes numériques :

```
colonnes_numeriques = df.select_dtypes(include='number').columns
for col in colonnes_numeriques:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    borne_basse = Q1 - 1.5 * IQR
    borne_haute = Q3 + 1.5 * IQR
    mediane = df[col].median()
    df.loc[(df[col] < borne_basse) | (df[col] > borne_haute), col] = mediane
```

Créer une table DATE : très très utile

```
Calendrier = ADDCOLUMNS ( CALENDARAUTO(),
    "Année", YEAR([Date]),
    "Mois", MONTH([Date]),
    "Jour", DAY([Date]),
    "Nom du Mois", FORMAT([Date], "MMM"),
    "Jour de la Semaine", WEEKDAY([Date]),
    "Nom du Jour de la Semaine", FORMAT([Date],
    "DDDD"), "Numéro de Semaine", WEEKNUM([Date]))
```

1 - ÉTAPES DÉTAILLÉES DE L'EDA

```
import pandas as pd
```

```
df = pd.read_csv('fichier.csv') # ou pd.read_excel(), pd.read_json(), etc.
df.head() # Aperçu des premières lignes
df.tail() # Aperçu des dernières lignes
df.shape # Dimensions (lignes, colonnes)
df.columns # Liste des noms de colonnes
df.info() # Types de données, nombre de valeurs non-nulles
df.dtypes # Types des colonnes
```

2 - Analyse statistique descriptive

```
df.describe() # Statistiques sur colonnes numériques
df.describe(include='object') # Statistiques sur colonnes catégorielles
df['colonne'].value_counts() # Comptage des modalités
df['colonne'].unique() # Modalités uniques
df['colonne'].nunique() # Nombre de modalités uniques
```

3 - Vérification des valeurs manquantes

```
df.isna().sum() # Nb de valeurs manquantes par colonne
df.isna().mean()*100 # Pourcentage de valeurs manquantes
df[df.isna().any(axis=1)] # Lignes contenant des valeurs manquantes
```

Traitements possibles

```
df['colonne'].fillna(valeur) # Remplacer par une valeur
df['colonne'].fillna(df['colonne'].mean()) # Moyenne
df['colonne'].fillna(df['colonne'].mode()[0]) # Valeur la plus fréquente
```

4 - Recherche de doublons

```
df.duplicated().sum() # Nb de doublons
df[df.duplicated()] # Affichage des doublons
df = df.drop_duplicates() # Suppression
```

5 - Analyse des types de variables

```
df.dtypes # Vérifier les types
df['date_colonne'] = pd.to_datetime(df['date_colonne']) # Conversion en datetime
df['colonne'] = df['colonne'].astype('category') # Pour colonnes catégorielles
```

6 - Détection des outliers (valeurs aberrantes)

```
df.boxplot(column='colonne')
```

```
Q1 = df['colonne'].quantile(0.25)
Q3 = df['colonne'].quantile(0.75)
IQR = Q3 - Q1
df[(df['colonne'] < Q1 - 1.5*IQR) | (df['colonne'] > Q3 + 1.5*IQR)]
```

7 - Analyse des corrélations

```
df.corr() # Matrice de corrélation
```

Visualisation

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
```

8 - Analyse univariée & bivariée

```
1 variable :
df['colonne'].hist() # Numérique
df['colonne'].value_counts().plot(kind='bar') # Catégorielle
```

```
2 variables :
sns.boxplot(x='colonne_catégorielle', y='colonne_numérique', data=df)
sns.scatterplot(x='col1', y='col2', data=df)
```

9 - Création de nouvelle variable

Exemple :

```
df['année'] = df['date_colonne'].dt.year
df['tranche_age'] = pd.cut(df['âge'], bins=[0,18,35,60,100], labels=['Enfant', 'Jeune', 'Adulte', 'Senior'])
```

10 - Exportation pour PowerBi

```
df.to_csv(".....csv", index=False)
df.to_parquet(".....parquet", index=False)
```

Avec compression : df.to_parquet(".....parquet", index=False, compression='gzip')

Passage sur Power Bi ...

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# AQUISITION FICHEIR
df = pd.read_csv("nom_du_fichier.csv")

# APERÇU GÉNÉRAL DES DONNÉES
print("Dimensions du dataset :", df.shape)
print("\nColonnes :", df.columns.tolist())
df.head(5)
df.info()
df.describe(include='all')

# ANALYSE DES VALEURS MANQUANTES
print("Valeurs manquantes par colonne :")
print(df.isnull().sum())
print("\nPourcentage de valeurs manquantes :")
print((df.isnull().mean()*100).round(2))

# GESTION DES VALEURS MANQUANTES (exemples)
# Exemple : remplissage par moyenne
df['colonne_numerique'] = df['colonne_numerique'].fillna(df['colonne_numerique'].mean())
# Exemple : suppression
df = df.dropna(subset=['colonne_importante'])

# RECHERCHE DE DOUBLONS
print("Nombre de doublons :", df.duplicated().sum())
df = df.drop_duplicates()

# VÉRIFICATION ET CONVERSION DES TYPES
df.dtypes
# Exemple de conversion
df['date_colonne'] = pd.to_datetime(df['date_colonne'])
df['colonne_categorielle'] = df['colonne_categorielle'].astype('category')

df[["fare_amount", "extra", "mta_tax", "tip_amount", "tolls_amount"]] = df[["fare_amount", "extra", "mta_tax", "tip_amount", "tolls_amount"]].astype("float16")

# ANALYSE STATISTIQUE UNIVARIÉE
# Pour les colonnes numériques
colonnes_num = df.select_dtypes(include='number').columns
df[colonnes_num].hist(bins=30, figsize=(15,10))
plt.suptitle("Distribution des variables numériques")
plt.show()

# Pour les colonnes catégorielles
colonnes_cat = df.select_dtypes(include='object').columns
for col in colonnes_cat:
    plt.figure()
    df[col].value_counts(normalize=True).plot(kind='bar')
    plt.title(f"Répartition de {col}")
    plt.ylabel("Fréquence")

# DÉTECTION DES OUTLIERS (Méthode IQR)
for col in colonnes_num:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    outliers = df[(df[col] < Q1 - 1.5 * IQR) | (df[col] > Q3 + 1.5 * IQR)]
    print(f"{col} : {outliers.shape[0]} outliers")
    # Boxplot pour visualisation
    plt.figure()
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot de {col}")

# ANALYSE BIVARIÉE / CROISÉE
# Corrélation des variables numériques
plt.figure(figsize=(12,8))
sns.heatmap(df[colonnes_num].corr(), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Matrice de corrélation")
plt.show()

# Exemple : scatterplot entre deux variables numériques
sns.scatterplot(x='colonne1', y='colonne2', data=df)
# Exemple : boxplot entre variable catégorielle et numérique
sns.boxplot(x='cat_var', y='num_var', data=df)

# FEATURE ENGINEERING (Création de variables)
# Exemple : extraire année et mois d'une date
df['année'] = df['date_colonne'].dt.year
df['mois'] = df['date_colonne'].dt.month
# Exemple : regrouper des tranches d'âge
df['tranche_age'] = pd.cut(df['age'], bins=[0, 18, 35, 60, 100], labels=['Enfant', 'Jeune', 'Adulte', 'Senior'])

# ✨ 12. SYNTHÈSE DES PREMIERS KPI POTENTIELS (exemples)
print("Revenus totaux :", df['montant'].sum())
print("Nombre de clients uniques :", df['client_id'].nunique())
print("Produit le plus vendu :", df['produit'].value_counts().idxmax())
print("Date de la première commande :", df['date'].min())
print("Date de la dernière commande :", df['date'].max())

# ✨ 13. EXPORT DU FICHIER NETTOYÉ POUR POWER BI
df.to_csv("df_clean_pour_powerbi.csv", index=False)

```

```

# Traitement 'day_of_week' et 'month'
df_clean['day_of_week'] = df_clean['started_at'].dt.day_name()
df_clean['month'] = df_clean['started_at'].dt.month_name()
df_clean.head()

#Fonction pour 'day_type'
def day_type(df):
    jours_weekend = ['Saturday', 'Sunday']
    df['day_type'] = df['day_of_week'].apply(lambda jour: 'Week-end' if jour in jours_weekend else 'Jour de semaine')
    return df

#Application de la fonction pour 'day_type'
df_clean = day_type(df_clean)
df_clean.head()

#Fonction pour 'season'
def season(df):
    winter = ['December', 'January', 'February']
    spring = ['March', 'April', 'May']
    summer = ['June', 'July', 'August']
    autumn = ['September', 'October', 'November']
    df['season'] = df['month'].apply(lambda m: ('Winter' if m in winter else 'Spring' if m in spring else 'Summer' if m in summer else 'Autumn'))
    return df

#Application de la fonction pour 'season'
df_clean = season(df_clean)
df_clean.head(5)

```

```

# Pour renommer toutes les colonnes
df_clean.rename(columns={"rideable_type":"type",
                        "started_at":"debut",
                        "ended_at":"fin",
                        "start_station_name":"station_depart",
                        "start_station_id":"id_station_depart",
                        "end_station_name":"station_arrivee",
                        "id_station_depart_arrivee":"id_station_arrivee",
                        "member_casual" : 'categorie_utilisateur',
                        "distance_travelled_km" : 'distance_km',
                        "ride_duration_s":"duree_trajet_seconde",
                        "day_of_week":"jour",
                        "month":"mois",
                        "day_type":"type_jour",
                        "start_lat":"lat_depart",
                        "start_lng":"lon_depart",
                        "end_lat":"lat_arrivee",
                        "end_lng":"lon_arrivee",
                        "season" : 'saison',
                        "route_type" : 'type_parcours',
                        "ride_duration_min":"duree_trajet_minutes"},

inplace=True)

```

Pour une régression linéaire ou régression Logistique

◇ 1. R^2 (coefficient de détermination)

☑ Ce que c'est :

C'est un score compris entre 0 et 1 (parfois négatif si le modèle est très mauvais) qui mesure la part de la variance des données expliquée par le modèle.

🧠 Interprétation :

$R^2 = 1 \rightarrow$ le modèle prédit parfaitement les valeurs

$R^2 = 0 \rightarrow$ le modèle ne fait pas mieux qu'une moyenne

$R^2 < 0 \rightarrow$ le modèle est pire que de toujours prédire la moyenne

📦 Exemple concret :

Tu veux prédire le prix d'un vélo d'occasion selon son âge. Si $R^2 = 0.85$, cela veut dire que 85 % des variations du prix peuvent être expliquées par l'âge.

◇ 2. MAE (Mean Absolute Error – Erreur Absolue Moyenne)

☑ Ce que c'est :

C'est la moyenne des erreurs absolues (en valeur positive, sans tenir compte du signe).

🧠 Interprétation :

MAE = 50 € $\rightarrow$ en moyenne, le modèle se trompe de 50 € par prédiction.

C'est simple à comprendre, car ça garde les unités de la variable cible.

📦 Exemple concret :

Si ton modèle prédit le prix d'un vélo et que le MAE est de 50 €, cela signifie qu'en moyenne, le prix prédit est à 50 € près du prix réel.

◇ 3. MSE (Mean Squared Error – Erreur Quadratique Moyenne)

donne plus d'importance au grosse erreur

☑ Ce que c'est :

Comme le MAE, mais ici on élève les erreurs au carré, puis on fait la moyenne.

Cela pénalise davantage les grosses erreurs.

🧠 Interprétation :

Utile si tu veux éviter les grosses erreurs

Mais ce n'est pas facile à interpréter directement, car ce n'est pas dans les mêmes unités que la variable prédite (prix par exemple).

📦 Exemple :

Tu prédis toujours un prix de vélo :

si le MSE est 2500, ça signifie que les erreurs au carré moyennes sont 2500, ce qui veut dire que l'erreur typique est $\sqrt{2500} = 50$ €.

◇ 4. RMSE (Root Mean Squared Error – Racine de l'erreur quadratique moyenne) ramène à la même unité que la valeur cible

☑ Ce que c'est :

C'est la racine carrée du MSE. On revient donc à la même unité que la variable cible.

Cela combine l'interprétabilité du MAE avec la sensibilité aux grosses erreurs du MSE.

🧠 Interprétation :

Si RMSE = 52 €, alors la moyenne des écarts entre les valeurs réelles et les prédictions est de 52 €, avec plus de pénalité sur les grosses erreurs qu'avec MAE.

📦 Exemple :

Tu prédis le prix de vélos, et ton RMSE est de 52 €. Cela signifie qu'en moyenne, ton modèle se trompe de 52 €, mais avec une plus grande importance donnée aux plus grosses erreurs.

`classification_report(y_true = y_test, y_pred = y_pred)`

Accuracy (précision) : Pourcentage global de bonnes prédictions, toutes classes confondues
Ton modèle a 86 % de bonnes prédictions sur l'ensemble des données

Precision : Proportion des prédictions positives qui sont réellement positives
Parmi toutes les fois où le modèle a prédit 1 (positif), combien étaient vraiment 1 ?
Quand ton modèle prédit 1, il se trompe environ 29 % du temps.

Recall (rappel) : Proportion des cas positifs réels qui sont correctement prédits
Parmi tous les vrais 1 (positifs réels), combien le modèle en a-t-il trouvé ?
Le modèle détecte seulement 54 % des vrais positifs. Il en rate presque 1 sur 2.

F1-score : Moyenne harmonique de la précision et du rappel

	Prédit 0		Prédit 1		
	Precision	Recall	F1-score	support	
0	0.89	0.94	0.91	26 652	
1	0.71	0.54	0.62	6 955	
Accuracy	0.89	0.74	0.86	33 607	
Macro avg	0.80	0.74	0.76	33 607	
Weight avg	0.85	0.81	0.85	33 607	